

ทดสอบ **Endpoints** ที่ต้องทำ

1. GET /products/{id}

The screenshot shows a REST client interface with a 'New Request' tab. The request method is 'GET' and the URL is 'http://localhost:8080/products/1'. The 'Send' button is visible. Below the URL bar, there are tabs for 'Docs', 'Params', 'Authorization', 'Headers 6', 'Body', 'Scripts', 'Settings', and 'Cookies'. The 'Params' tab is active, showing a 'Query Params' table with columns 'Key', 'Value', 'Description', 'Bulk Edit', and '...'. The table is currently empty.

Below the 'Params' tab, there are tabs for 'Body', 'Cookies', 'Headers 2', and 'Test Results'. The 'Body' tab is active, showing a JSON response with a status of '200'. The JSON response is as follows:

```
1  {
2    "id": "1",
3    "name": "iPhone 15 Pro (673380066-4 Sapphanyu SEC 1)",
4    "category": "Electronics",
5    "brand": "Apple",
6    "stock": 50,
7    "price": 39900.0,
8    "discountType": "MEMBER",
9    "discountedPrice": 35910.0
10 }
```

2. GET /products

The screenshot shows a REST client interface with a 'New Request' tab. The request method is 'GET' and the URL is 'http://localhost:8080/products'. The response status is '200'.

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body (JSON)

```
{
  "id": "2",
  "name": "MacBook Air M3",
  "category": "Electronics",
  "brand": "Apple",
  "stock": 20,
  "price": 49900.0,
  "discountType": "NONE",
  "discountedPrice": 49900.0
},
{
  "id": "3",
  "name": "Samsung Galaxy S24",
  "category": "Electronics",
  "brand": "Samsung",
  "stock": 30,
  "price": 29900.0,
  "discountType": "SEASONAL",
  "discountedPrice": 23920.0
}
```

Console

★ The coast is clear

Globals Vault Tools

3. POST /products

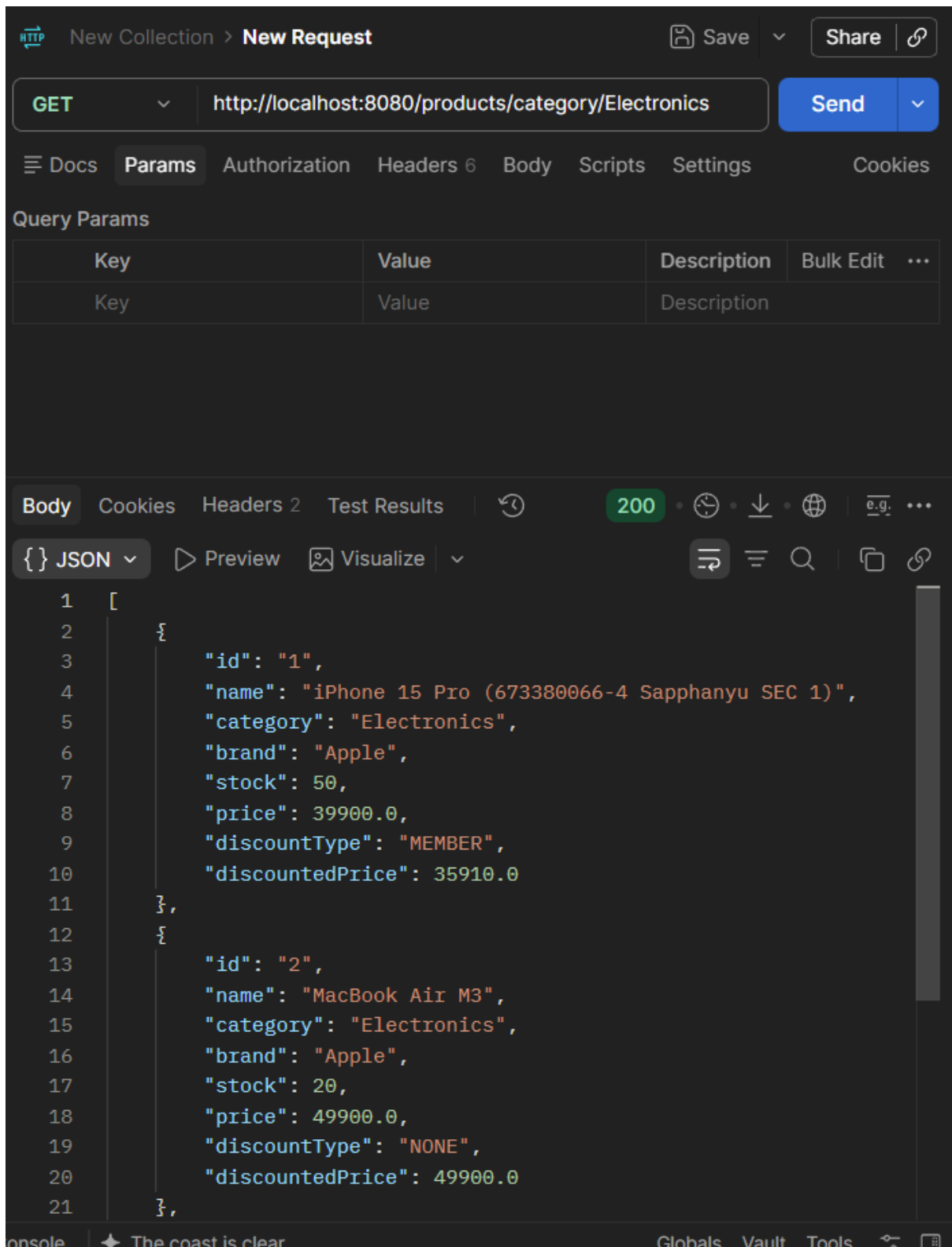
The screenshot displays a REST client interface with the following components:

- Top Bar:** Includes tabs for HTTP, Runner, and New Collection. The environment is set to "No Environment".
- Request Section:**
 - Method:** POST
 - URL:** http://localhost:8080/products
 - Buttons:** Save, Share, and Send.
 - Tabs:** Docs, Params, Authorization, Headers 8, Body (selected), Scripts, Settings, Cookies.
 - Body Format:** raw (selected), JSON.
 - Options:** Schema, Beautify.
- Request Body (JSON):**

```
1  {
2    "name": "Sony WH-1000XM5",
3    "category": "Audio",
4    "brand": "Sony",
5    "stock": 15,
6    "price": 11900.0,
7    "discountType": "MEMBER"
8  }
```
- Response Section:**
 - Body Tab:** Selected.
 - Response Format:** {} JSON.
 - Buttons:** Preview, Visualize.
 - Status:** 200.
 - Response Body (JSON):**

```
1  {
2    "id": "0c5be6f6-2ef0-4354-b94b-5e1e8f5b9013",
3    "name": "Sony WH-1000XM5",
4    "category": "Audio",
5    "brand": "Sony",
6    "stock": 15,
7    "price": 11900.0,
8    "discountType": "MEMBER",
9    "discountedPrice": 10710.0
10 }
```

4.GET /products/category/Electronics



The screenshot shows a REST client interface with the following details:

- Request:** Method: GET, URL: `http://localhost:8080/products/category/Electronics`
- Response:** Status: 200, Content-Type: JSON
- JSON Response:**

```
[
  {
    "id": "1",
    "name": "iPhone 15 Pro (673380066-4 Sapphanyu SEC 1)",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 50,
    "price": 39900.0,
    "discountType": "MEMBER",
    "discountedPrice": 35910.0
  },
  {
    "id": "2",
    "name": "MacBook Air M3",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 20,
    "price": 49900.0,
    "discountType": "NONE",
    "discountedPrice": 49900.0
  }
]
```

```
[
  {
    "id": "1",
```

```
"name": "iPhone 15 Pro (673380066-4 Sapphanyu SEC 1)",  
  
"category": "Electronics",  
  
"brand": "Apple",  
  
"stock": 50,  
  
"price": 39900,  
  
"discountType": "MEMBER",  
  
"discountedPrice": 35910  
},  
{  
  
  "id": "2",  
  
  "name": "MacBook Air M3",  
  
  "category": "Electronics",  
  
  "brand": "Apple",  
  
  "stock": 20,  
  
  "price": 49900,  
  
  "discountType": "NONE",  
  
  "discountedPrice": 49900  
},  
{  
  
  "id": "3",  
  
  "name": "Samsung Galaxy S24",  
  
  "category": "Electronics",  
  
  "brand": "Samsung",  
  
  "stock": 30,  
  
  "price": 29900,
```

"discountType": "SEASONAL",

"discountedPrice": 23920

}

]

5. GET /products/1/price

The screenshot shows a REST client interface with the following components:

- Top Bar:** "New Collection > New Request", "Save", and "Share" buttons.
- Request Bar:** Method "GET" and URL "http://localhost:8080/products/1/price". A "Send" button is on the right.
- Params Tab:** Active tab showing "Query Params".
- Query Params Table:**

Key	Value	Description	Bulk Edit	...
Key	Value	Description		
- Response Section:**
 - Body Tab:** Active tab showing the response.
 - Status:** "200" in a green box.
 - Response Type:** "{ } JSON".
 - Actions:** "Preview", "Visualize", and other utility icons.
 - Response Content:**

```
1 35910.0
```

6. DELETE /products/{id}

ก่อนลบ

The screenshot shows a REST client interface with a 'New Request' tab. The request method is 'GET' and the URL is 'http://localhost:8080/products/0c5be6f6-2ef0-4354-t...'. The 'Send' button is visible. Below the request details, the 'Params' tab is active, showing a table for 'Query Params'.

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Below the 'Params' tab, the 'Body' tab is active, showing a JSON response with a status of 200. The JSON data is as follows:

```
1  {
2    "id": "0c5be6f6-2ef0-4354-b94b-5e1e8f5b9013",
3    "name": "Sony WH-1000XM5",
4    "category": "Audio",
5    "brand": "Sony",
6    "stock": 15,
7    "price": 11900.0,
8    "discountType": "MEMBER",
9    "discountedPrice": 10710.0
10 }
```


หลังลบ

 New Collection > **New Request**

 Save

 Share

DELETE

http://localhost:8080/products/0c5be6f6-2ef0-4354-t...

Send

Docs

Params

Authorization

Headers 6

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

Cookies

Headers 1

Test Results



200









...

Raw

Preview

Visualize









1

1. Reactive Programming vs Blocking — ต่างกันอย่างไร

คุณลักษณะ

Blocking I/O (Spring MVC / Servlet)

Reactive Non-blocking (Spring WebFlux / Netty)

รูปแบบการทำงาน

Synchronous & Blocking

Asynchronous & Non-blocking (Event-driven)

การใช้ Thread

1 Request ต่อ 1 Thread (Thread per Request)

ถ้ามี 200 requests พร้อมกัน ต้องใช้ 200 threads

ใช้ Threads จำนวนน้อยคงที่ (Event Loop Threads)

มักเท่ากับจำนวน CPU Cores ทำงานหมุนเวียนรับหลายหมื่น requests ได้

พฤติกรรมขณะรอ I/O

เมื่อ Thread เรียกอ่าน DB หรือ Network Thread จะ "หยุดรอ (Block/Idle)" ทำงานอื่นไม่ได้

Thread จะ "ไม่หยุดรอ" ส่ง I/O แล้วปล่อยให้ Thread ไปทำงานให้ Request อื่นต่อทันที เมื่อ I/O เสร็จจะมี Event Callback กลับมา

การรองรับ Concurrency

จำกัดตามขนาด Thread Pool ถ้า Request เข้ามาล้น Thread จะเกิดคอขวดและ Out of Memory ได้ง่าย

รองรับ Concurrency สูงมาก (High Throughput) ใช้ทรัพยากร Memory และ CPU ค่อนข้าง

โครงสร้าง Return

ส่งคืน Object ข้อมูลโดยตรง เช่น Product, List<Product>

ส่งคืน Reactive Streams Publisher เช่น Mono<Product>, Flux<Product>

สรุปเปรียบเทียบ:

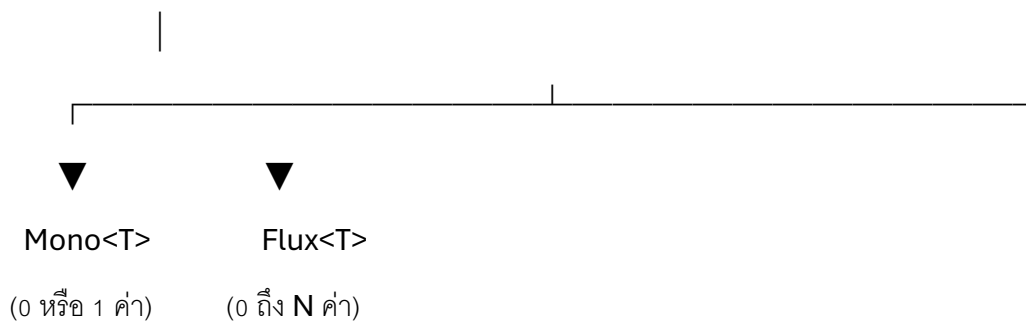
Blocking เปรียบเหมือนการต๋อคิวสั่งอาหารแล้วลูกค้ายืนรอตรงหน้าเคาน์เตอร์จนกว่าอาหารจะเสร็จ (พนักงาน 1 คนรับลูกค้าได้ทีละคน)

Reactive เปรียบเหมือนการสั่งอาหารแล้วรับบัตรคิว (Buzzer) ไปนั่งรอ พนักงานก็รับออเดอร์ลูกค้าคนถัดไปได้ทันที เมื่ออาหารเสร็จสัญญาณจะดังเรียกให้มารับ

2. Mono vs Flux — ใช้กรณีไหน และทำไม

ทั้ง **Mono** และ **Flux** เป็น Implementations ของอินเทอร์เฟซ **Publisher<T>** ใน Project Reactor ซึ่ง เป็นแกนหลักของ Spring WebFlux

Publisher<T> (ตัวปล่อย Data Stream)



1) Mono<T> (ปล่อยข้อมูล 0 ถึง 1 ตัว)

ใช้กรณีไหน:

เมื่อการทำงานนั้นให้ผลลัพธ์เพียง ชิ้นเดียว (Single value) หรือ ไม่มีผลลัพธ์เลย (Empty)

เช่น ค้นหาตาม ID (findById), การบันทึก/แก้ไข (save, update), หรือลบข้อมูลที่ไม่ต้องคืนค่า (delete คืน Mono<Void>)

ทำไมถึงใช้: เพื่อบอกระบบและผู้บริโภคข้อมูลว่า Stream นี้จะส่งสัญญาณ onNext(item) เพียงครั้งเดียวแล้ว ตามด้วย onComplete() ทันที ทำให้ประมวลผลได้รวดเร็วและใช้หน่วยความจำน้อย

2) Flux<T> (ปล่อยข้อมูล 0 ถึง N ตัว)

ใช้กรณีไหน:

เมื่อการทำงานคืนค่าเป็น ชุดข้อมูลหลายตัว (Collection/Stream) หรือ Real-time Event Stream

เช่น การดึงรายการทั้งหมด (findAll), การค้นหาตามหมวดหมู่ (findByCategory), หรือ Server-Sent Events (SSE)

ทำไมถึงใช้: ข้อมูลจะถูกทยอยส่งทีละชิ้น (Streaming) ผ่านสัญญาณ `onNext(item)` ซ้ำๆ จนกว่าจะหมดแล้วส่ง `onComplete()` โดยไม่จำเป็นต้องรอโหลดข้อมูลทั้งหมดขึ้น Memory ให้ครบก่อนเหมือน `List<T>`

3. WebClient — อธิบาย Method Chain

`WebClient` คือ HTTP Client แบบ Non-blocking และ Reactive ของ Spring WebFlux ซึ่งเข้ามาแทนที่ `RestTemplate` (ที่เป็น Blocking)

ตัวอย่างโค้ด Method Chain:

```
client.get()
    .uri("/products/{id}", id)
    .retrieve()
    .bodyToMono(Product.class);
```

อธิบายการทำงานแต่ละ Method:

`.get()` / `.post()` / `.delete()` (HTTP Method):

เป็นการระบุ HTTP Method ที่ต้องการส่งคำขอ เช่น `.get()` สำหรับดึงข้อมูล, `.post()` สำหรับสร้างข้อมูลใหม่

`.uri("/products/{id}", id)` (URI Specification):

ใช้กำหนด Path Endpoint และ Parameter ที่ต้องการส่งไป โดยระบบจะนำค่าตัวแปร `id` ไปแทนที่ใน `{id}` ให้อัตโนมัติอย่างปลอดภัย

`.bodyValue(product)` (ในกรณี POST/PUT):

ใช้แนบ Object ข้อมูลเข้าไปใน Request Body เพื่อส่งไปยัง Server

`.retrieve()` (Response Execution):

เป็นคำสั่งเริ่มส่ง HTTP Request ออกไป และรอรับ HTTP Response Headers/Status เพื่อเตรียมแปลงเนื้อหา Response Body

`.bodyToMono(T.class)` / `.bodyToFlux(T.class)` (Body Extraction):

`.bodyToMono(Product.class)`: แปลง JSON Body ที่ตอบกลับมาให้เป็น `Mono<Product>` (สำหรับข้อมูล 1 ชิ้น)

.bodyToFlux(Product.class): แปลง JSON Array ที่ตอบกลับมาให้เป็น Flux<Product> (สำหรับข้อมูลหลายชิ้น)

4. Source Code พร้อม Comment อธิบายอย่างละเอียด

4.1 ProductRepository.java

```
package com.example.lab10.repository;
```

```
import com.example.lab10.model.Product;
```

```
import reactor.core.publisher.Flux;
```

```
import reactor.core.publisher.Mono;
```

```
import java.util.Map;
```

```
import java.util.concurrent.ConcurrentHashMap;
```

```
/**
```

```
 * ProductRepository — In-memory Reactive Repository
```

```
 * จำลองการเก็บข้อมูลลงใน ConcurrentHashMap แบบ Thread-safe
```

```
 * และคืนค่าผลลัพธ์เป็น Mono/Flux เพื่อให้เป็น Non-blocking API
```

```
 */
```

```
public class ProductRepository {
```

```
    // ใช้ ConcurrentHashMap เก็บข้อมูลชั่วคราวในหน่วยความจำ
```

```
    private final Map<String, Product> store = new ConcurrentHashMap<>();
```

```
    // Constructor: กำหนด Seed Data เริ่มต้น 3 รายการ
```

```
    public ProductRepository() {
```

```
        // รายการที่ 1: กำหนดชื่อพร้อมรหัสนักศึกษาและ Section
```

```
        store.put("1", new Product("1", "iPhone 15 Pro (673380066-4 Sapphanyu SEC 1)",
```

```

        "Electronics", "Apple", 50, 39900.0, "MEMBER"));

store.put("2", new Product("2", "MacBook Air M3",

        "Electronics", "Apple", 20, 49900.0, "NONE"));

store.put("3", new Product("3", "Samsung Galaxy S24",

        "Electronics", "Samsung", 30, 29900.0, "SEASONAL"));

}

// 1. ค้นหาสินค้าตาม ID: คืนค่าเป็น Mono<Product> (ถ้าไม่พบ คืน Mono.empty())
public Mono<Product> findById(String id) {
    Product product = store.get(id);
    return product != null ? Mono.just(product) : Mono.empty();
}

// 2. ดึงสินค้าทั้งหมด: ใช้ Flux.fromIterable แปลง Collection เป็น Flux<Product>
public Flux<Product> findAll() {
    return Flux.fromIterable(store.values());
}

// 3. บันทึกสินค้า: ใส่ข้อมูลลง Map และห่อผลลัพธ์กลับด้วย Mono.just
public Mono<Product> save(Product product) {
    store.put(product.getId(), product);
    return Mono.just(product);
}

// 4. ลบสินค้าตาม ID: ลบออกจาก Map และคืน Mono.empty() ในรูปแบบ Mono<Void>
public Mono<Void> deleteById(String id) {
    store.remove(id);
    return Mono.empty();
}

```

```
}
```

// 5. กรองสินค้าตาม **Category**: ดึงทั้งหมดแล้วใช้ **Operator .filter()** กรองเฉพาะหมวดหมู่ที่ต้องการ

```
public Flux<Product> findByCategory(String category) {  
    return findAll()  
        .filter(p -> p.getCategory() != null &&  
p.getCategory().equalsIgnoreCase(category));  
}  
}
```

4.2 ProductService.java

```
package com.example.lab10.service;
```

```
import com.example.lab10.model.Product;
```

```
import com.example.lab10.repository.ProductRepository;
```

```
import org.springframework.stereotype.Service;
```

```
import reactor.core.publisher.Flux;
```

```
import reactor.core.publisher.Mono;
```

```
/**
```

```
 * ProductService — Business Logic Layer
```

```
 * ทำหน้าที่ประมวลผลตรรกะทางธุรกิจก่อนส่งต่อหรือรับค่าจาก Repository
```

```
 */
```

```
@Service
```

```
public class ProductService {
```

```
    // ทำ Dependency Injection ผ่าน Constructor
```

```
private final ProductRepository repository;
```

```
public ProductService(ProductRepository repository) {  
    this.repository = repository;  
}
```

// 1. ดึงสินค้าตาม ID: หากไม่พบข้อมูล ให้เปลี่ยน Stream เป็น Mono.error (Non-blocking exception)

```
public Mono<Product> getByld(String id) {  
    return repository.findByld(id)  
        .switchIfEmpty(Mono.error(new RuntimeException("Product not found: " + id)));  
}
```

// 2. ดึงสินค้าทั้งหมด

```
public Flux<Product> getAll() {  
    return repository.findAll();  
}
```

// 3. บันทึกสินค้า: หากไม่มีการระบุ ID มาด้วย จะทำการสุ่ม UUID ให้อัตโนมัติ

```
public Mono<Product> save(Product product) {  
    if (product.getld() == null || product.getld().isBlank()) {  
        product.setld(java.util.UUID.randomUUID().toString());  
    }  
    return repository.save(product);  
}
```

// 4. ลบสินค้าตาม ID

```
public Mono<Void> delete(String id) {  
    return repository.deleteByld(id);  
}
```


// 5. ค้นหาสินค้าตามหมวดหมู่

```
public Flux<Product> getByCategory(String category) {  
    return repository.findByCategory(category);  
}
```

// 6. คำนวณราคาหลังหักส่วนลด: ดึง Product มาแล้วใช้ .map() แปลงเป็นตัวเลขราคาหลังหักส่วนลด

```
public Mono<Double> getDiscountedPrice(String id) {  
    return getById(id)  
        .map(Product::getDiscountedPrice);  
}  
}
```

4.3 ProductController.java

```
package com.example.lab10.controller;
```

```
import com.example.lab10.model.Product;
```

```
import com.example.lab10.service.ProductService;
```

```
import org.springframework.web.bind.annotation.*;
```

```
import reactor.core.publisher.Flux;
```

```
import reactor.core.publisher.Mono;
```

```
/**
```

```
 * ProductController — Reactive REST Controller
```

```
 * ทุก Endpoint ส่งคืน Mono หรือ Flux เพื่อให้ Netty ประมวลผลแบบ Non-blocking ตลอดกระบวนการ
```

```
 */
```

```
@RestController
```

```
@RequestMapping("/products")
```

```
public class ProductController {
```

```
    private final ProductService service;
```

```
    public ProductController(ProductService service) {
```

```
        this.service = service;
```

```
    }
```

```
    // 1. GET /products/{id} : ดึงข้อมูลสินค้า 1 ชิ้น คืนค่า Mono<Product>
```

```
    @GetMapping("/{id}")
```

```
    public Mono<Product> getById(@PathVariable String id) {
```

```
        return service.getById(id);
```

```
    }
```

```
    // 2. GET /products : ดึงข้อมูลสินค้าทั้งหมด คืนค่า Flux<Product>
```

```
    @GetMapping
```

```
    public Flux<Product> getAll() {
```

```
        return service.getAll();
```

```
    }
```

```
    // 3. POST /products : รับ JSON จาก Request Body แล้วบันทึกสินค้าใหม่ คืนค่า  
    Mono<Product>
```

```
    @PostMapping
```

```
    public Mono<Product> save(@RequestBody Product product) {
```

```
        return service.save(product);
```

```
    }
```

```
    // 4. DELETE /products/{id} : ลบสินค้าตาม ID คืนค่า Mono<Void> (200 OK เปล่า)
```

```
    @DeleteMapping("/{id}")
```

```
public Mono<Void> delete(@PathVariable String id) {  
    return service.delete(id);  
}
```

// 5. GET /products/category/{category} : ดึงสินค้าเฉพาะในหมวดหมู่ที่ระบุ คืนค่า
Flux<Product>

```
@GetMapping("/category/{category}")  
public Flux<Product> getByCategory(@PathVariable String category) {  
    return service.getByCategory(category);  
}
```

// 6. GET /products/{id}/price : ดึงราคาหลังส่วนลดของสินค้า คืนค่า Mono<Double>

```
@GetMapping("/{id}/price")  
public Mono<Double> getDiscountedPrice(@PathVariable String id) {  
    return service.getDiscountedPrice(id);  
}  
}
```